Universal Binary Principle (UBP) Framework v3.5

Comprehensive Instruction Manual

Author: Euan Craig, New Zealand | Date: 14 November 2025

Executive Summary

UBP 3.5 represents a fundamental paradigm shift in the Universal Binary Principle, evolving from a system that measures coherence to a system where **computation IS coherence**. This version introduces the `coherence_substrate.py` module, a zero-dependency Python implementation that establishes a new foundation for all UBP operations. Every numerical value is now a `CoherenceState` object, a self-aware entity that carries its own value, coherence, and operational history.

This architectural revolution supersedes the dependency-heavy, multi-module approach of previous versions. Complex functionalities like error correction and recursive field dynamics, which previously required dedicated modules (`glr_base.py` , `CARFE`), now emerge naturally from the inherent geometric properties of the coherence substrate itself. The result is a dramatically simplified, more powerful, and philosophically pure implementation of the UBP.

Key Achievements of UBP 3.5:

- **Coherence-Native Paradigm**: Computation is no longer performed on raw numbers; it is performed on `CoherenceState` objects that intrinsically manage their own quality.
- **Zero Dependencies**: The entire UBP 3.5 system runs on pure Python, requiring no external libraries like NumPy or SciPy, making it universally portable and maximally trustworthy.
- **Unified Geometric Error Correction**: A single `geometric_error_correction.py` module replaces a suite of older error correction systems, providing self-healing capabilities inherent to the substrate.
- **Emergent Field Dynamics**: The new `advanced_modules/field_dynamics.py` replaces the complex `CARFE` module, demonstrating that advanced physical phenomena like Zitterbewegung and recursive evolution are emergent properties of the coherence substrate.

- **Radical Simplification**: The system has been streamlined from over 70 files in UBP 3.4 to 24 core modules in UBP 3.5, increasing clarity and maintainability without sacrificing any capability.

Table of Contents

---

1. A New Philosophy: Computation as Coherence {#philosophy}

Previous versions of the UBP framework treated computation and coherence as separate concerns. First, a numerical operation was performed (e.g., multiplication, addition). Then, a separate process was used to measure or correct the resulting coherence. This approach, while effective, created a complex, multi-layered system where the integrity of a value was external to the value itself.

UBP 3.5 introduces a revolutionary and far more elegant paradigm: **the substrate IS the system**. There is no separation between a value and its quality. Every number, every constant, and every result is a `CoherenceState`—an object that encapsulates not just its numerical value but its entire history of coherence, uncertainty, and refinement.

> In UBP 3.5, we no longer ask, "What is the coherence of this value?" Instead, the value itself tells us its coherence. We no longer apply error correction as an afterthought; operations are intrinsically self-correcting. This is the principle of **computation as coherence**.

This shift has profound implications:

- **Trust and Transparency**: Because the system has zero external dependencies and every operation tracks its own quality, the entire computational chain is transparent and verifiable from first principles.
- **Simplicity and Power**: Complex behaviors that previously required specialized, high-maintenance modules now emerge naturally from the fundamental geometry of the coherence substrate. The system is simultaneously simpler and more powerful.
- **Philosophical Purity**: UBP 3.5 is a more direct and pure implementation of the Universal Binary Principle. It treats information not as a static quantity to be measured, but as a dynamic, self-aware entity that actively maintains its own integrity.

This manual is designed to guide you through this new way of thinking and operating within the UBP framework. It is not just an update; it is an introduction to a new computational philosophy.

---

2. Quick Start: Your First Coherence-Native Calculation {#quick-start}

Getting started with UBP 3.5 is simpler than ever before, thanks to the zero-dependency architecture. All you need is a standard Python 3.11+ environment.

Installation

There are no external packages to install. Simply clone the repository and you are ready to begin.

Bash

```
# Clone the UBP 3.5 repository
git clone https://github.com/DigitalEuan/UBP_Repo.git

# Navigate to the UBP 3.5 directory
cd UBP_Repo/ubp_3.5

# Verify the system is operational by running the validation script
python3.11 validate_system.py
```

Upon successful validation, you will see the message: `🎉 All UBP 3.5 Core Systems Validated and Operational! 🎉`

Your First UBP 3.5 Calculation

The following example demonstrates the fundamental difference in UBP 3.5. Notice how we operate directly on `CoherenceState` objects and how they inherently track their own quality.

Python

```python
from coherence_substrate import CoherenceState, Y_CONSTANT, Y_INVERSE
from system_constants import UBPConstants

# 1. Work with CoherenceState objects directly
# These are not floats; they are self-aware computational entities.
print(f"Y_CONSTANT is of type: {type(Y_CONSTANT )}")
print(f"Y_CONSTANT: {Y_CONSTANT}")

# 2. Perform arithmetic operations
# The overloaded operators automatically handle coherence tracking.
product = Y_CONSTANT * Y_INVERSE

print(f"\nProduct (Y * 1/Y): {product}")
print(f"Closure Error: {abs(product.value - 1.0):.2e}")

# 3. Create your own CoherenceState
# Start with a value and an initial coherence (NRCI).
initial_energy = CoherenceState(value=1e12, nrci=0.999)
print(f"\nInitial Energy: {initial_energy}")

# 4. Apply a coherence-preserving transformation
# The '*' operator is a geometric transformation, not just multiplication.
refined_energy = initial_energy * Y_CONSTANT
print(f"Refined Energy: {refined_energy}")

# 5. Observe how coherence evolves
# The NRCI of the result is a product of the input coherences.
expected_nrci = initial_energy.nrci * Y_CONSTANT.nrci
print(f"Expected NRCI: {expected_nrci:.10f}")
print(f"Actual NRCI:   {refined_energy.nrci:.10f}")
```

This simple example reveals the power of the new paradigm. Every variable is a rich object containing its full computational history, and every operation is a geometric transformation that preserves and tracks coherence automatically.

---

3. What's New in 3.5: The Paradigm Shift {#whats-new}

UBP 3.5 is not an incremental update; it is a complete architectural refactoring that prioritizes philosophical purity and computational integrity. The changes from UBP 3.4 are

profound and touch every aspect of the system. Understanding these changes is key to leveraging the full power of the new coherence-native paradigm.

From Dependency-Heavy to Zero-Dependency

The most significant practical change is the elimination of all external libraries. UBP 3.4 relied on `NumPy` and `SciPy` for numerical operations, array handling, and scientific functions. UBP 3.5 has no such dependencies. It is implemented in pure Python.

Aspect	UBP 3.4 (Legacy)	UBP 3.5 (Coherence-Native)
Dependencies	NumPy, SciPy	**None** (Pure Python)
Portability	Requires environment setup	Runs anywhere Python runs
Trust	Trust in external libraries	Trust in verifiable, self-contained code

This makes UBP 3.5 maximally portable, auditable, and robust. The entire system can be understood and verified without needing to trust external, pre-compiled binaries.

The `CoherenceState` Object: A Fundamental Change

In UBP 3.4, all calculations were performed on standard Python floats. In UBP 3.5, all numerical values are encapsulated within the `CoherenceState` class. This is the cornerstone of the new paradigm.

> A `CoherenceState` is not just a number. It is a self-aware computational entity that holds its value, its Non-Random Coherence Index (NRCI), its uncertainty, and a history of the operations that created it. Arithmetic operations like multiplication and addition are overloaded to automatically and correctly propagate these properties through every calculation.

Unified and Emergent Functionality

The shift to a coherence-native substrate has allowed for a radical simplification of the system architecture. Complex functionalities that required dedicated modules in UBP 3.4 now emerge naturally from the interactions within the substrate.

Functionality	UBP 3.4 Implementation	UBP 3.5 Implementation
Error Correction	`glr_base.py`, `level_7_global_golay.py`, `p_adic_correction.py`	A single `geometric_error_correction.py` whose functions are inherent to `CoherenceState` operations.
Field Dynamics	`advanced_modules/carfe.py` (838 lines, complex)	A new `advanced_modules/field_dynamics.py` that achieves the same results as an emergent property of geometric operations.
Core Constants	Floats defined in `y_constants.py`	`CoherenceState` objects, carrying their own perfect coherence.
System Size	~70+ Python files	**24 core modules**, a 65% reduction.

This consolidation is not a reduction in capability. On the contrary, it represents a significant increase in power and elegance, as complex behaviors are no longer bolted on but emerge from a simple, consistent set of rules.

---

4. Core Concepts of the Coherence Substrate {#core-concepts}

To truly understand UBP 3.5, one must understand its foundation: the `coherence_substrate.py` module. This single file contains the complete logic for the new paradigm. At its heart is the `CoherenceState` class, which redefines what a "number" is within the UBP framework.

The `CoherenceState` Class

A `CoherenceState` is the fundamental data type of UBP 3.5. It replaces the standard `float` for all calculations. Each instance of this class is a rich object with several key attributes that track its quality and history.

Attribute	Type	Description
`value`	`float`	The numerical value of the state.
`nrci`	`float`	The **Non-Random Coherence Index** (0 to 1), representing the state's quality.
`log_nrci`	`float`	The natural logarithm of the NRCI, used for stable error accumulation.
`uncertainty`	`float`	The calculated uncertainty, derived from the NRCI.
`history`	`str`	A string describing the operation that created this state.

Example: Creating a `CoherenceState`

Python

```python
from coherence_substrate import CoherenceState

# Create a state with a value of pi and perfect coherence
pi_state = CoherenceState(value=3.14159, nrci=1.0, history="Initial: pi")

print(pi_state)
# Output: CoherenceState(value=3.14159, nrci=1.0, uncertainty=0.0,
history='Initial: pi')
```

Coherence-Preserving Operations

The true power of the `CoherenceState` comes from its overloaded arithmetic operators (`+` , `-` , `*` , `/` , `**`). When you perform an operation between two `CoherenceState` objects, you are not just calculating a new value; you are creating a new `CoherenceState` with a correctly propagated coherence.

The Golden Rule of Coherence Propagation: When two `CoherenceState` objects are multiplied, their NRCIs are also multiplied. To make this process numerically stable and physically meaningful, the system operates on the logarithms of the NRCIs.

> log_nrci_new = log_nrci_A + log_nrci_B

This means that error accumulation is additive in the logarithmic space, which correctly models the degradation of information in a physical system. A new `CoherenceState` is then formed with `nrci_new = exp(log_nrci_new)`.

Self-Healing and Coherence Restoration

UBP 3.5 is not just a passive system for tracking error; it is an active system that seeks to preserve coherence. This is achieved through the `geometric_error_correction` module, which provides functions to restore a `CoherenceState` to a higher quality.

The `restore_coherence()` function is a key example. It uses the geometric properties of the Y constant to project a degraded `CoherenceState` back towards a state of perfect coherence. This is the practical implementation of the system's self-healing capabilities.

Python

```python
from coherence_substrate import CoherenceState, Y_CONSTANT
from geometric_error_correction import restore_coherence

# 1. Create a degraded state
degraded_state = CoherenceState(value=100.0, nrci=0.9)
print(f"Original: {degraded_state}")

# 2. Apply a transformation that further degrades it
transformed_state = degraded_state * CoherenceState(value=0.5, nrci=0.9)
print(f"Transformed: {transformed_state}")

# 3. Restore its coherence using geometric projection
restored_state, _ = restore_coherence(transformed_state, Y_CONSTANT)
print(f"Restored: {restored_state}")
```

This example demonstrates a complete cycle: a state is created, it degrades through operations, and its coherence is then actively restored. This dynamic process of degradation and restoration is the essence of computation in UBP 3.5.

---

5. System Architecture: A Unified Framework {#architecture}

The architecture of UBP 3.5 is a testament to the power of its core philosophy. By embedding intelligence directly into the substrate, the system achieves a clean, layered, and logical structure that is significantly simpler than its predecessors. The framework can be understood as a series of layers, each building upon the capabilities of the one below it.

Layer	Core Modules	Purpose
4. Advanced Dynamics	`field_dynamics.py`	Models complex, emergent physical phenomena like recursive field evolution (formerly CARFE).
3. Physical Realms	`quantum_realm.py`, `gravitational_realm.py`, etc. (9 total)	Applies the UBP framework to specific physical domains, calculating energies and state transitions.
2. Computational Core	`soc_energy.py`, `geometric_error_correction.py`, `observer_framework.py`	Implements the core UBP logic: SOC energy calculations, coherence restoration, and observer dynamics.
1. State Management	`state.py`, `toggle_ops.py`, `tgic.py`	Manages the discrete, bit-level state of the UBP system (OffBits).
0. Foundation	`coherence_substrate.py`, `y_constants.py`, `system_constants.py`	Defines the fundamental `CoherenceState` object and the core mathematical constants of the universe.

Layer 0: The Foundation - The Substrate Itself

This is the bedrock of UBP 3.5. The `coherence_substrate.py` module defines the `CoherenceState` and the rules for its interaction. It is the engine of the entire system. The `y_constants.py` and `system_constants.py` modules populate this substrate with the fundamental, universal constants (like Y, Pi, and the Golden Ratio), all defined as `CoherenceState` objects with perfect intrinsic coherence.

Layer 1: State Management

Building on the substrate, the state management layer handles the discrete, binary nature of the UBP. The `state.py` module defines the `OffBit`, the fundamental unit of UBP information, now built from `CoherenceState` objects. `toggle_ops.py` and `tgic.py` define the rules for how these states can interact and change, ensuring that all state transitions adhere to the system's geometric constraints.

Layer 2: The Computational Core

This layer contains the primary physics and logic of the UBP. The `soc_energy.py` module calculates the energy required to maintain a coherent state (Simplified Observer Coherence). The `geometric_error_correction.py` module provides the tools for self-healing and coherence restoration. Finally, the `observer_framework.py` models the role of the observer, whose very act of observation is a computational process with a defined geometric cost.

Layer 3: The Physical Realms

This is where the abstract principles of the UBP are applied to concrete physical problems. Each of the nine realm modules (`quantum_realm.py` , `gravitational_realm.py` , etc.) uses the computational core to model phenomena specific to its domain. For example, the `quantum_realm` calculates the energy of quantum states, while the `gravitational_realm` models the energy of spacetime curvature. In UBP 3.5, all these calculations are performed using coherence-native objects, providing a new level of insight into the quality and integrity of the results.

Layer 4: Advanced Dynamics

At the highest level, the system can model complex, emergent behaviors. The `field_dynamics.py` module is the prime example, demonstrating how the recursive evolution of fields—a concept that required the highly complex and specialized `CARFE` module in UBP 3.4—can be modeled as a natural, emergent property of the underlying geometric operations in the coherence substrate. This layer shows the ultimate power of the UBP 3.5 paradigm: complexity arises from simplicity. \n---\n\n## 6. Module Reference: The Building Blocks {#modules}\n\nThis section provides a detailed reference for the most critical modules in the UBP 3.5 framework. Understanding these modules is essential for using, extending, and contributing to the system.\n\n### `coherence_substrate.py` (The Foundation)\n\nThis is the single most important file in UBP 3.5. It contains the complete implementation of the coherence-native paradigm and has **zero dependencies**.\n\n**Core Component: `CoherenceState` Class**\n\n* **`CoherenceState(value: float, nrci: float = 1.0, log_nrci: float = 0.0, history: str = '')`** \n The constructor for the fundamental data type. It is recommended to provide either `nrci` or `log_nrci` , but not both. If both are provided, `log_nrci` takes precedence.\n\n* **Arithmetic Operators**\n The class overloads all standard arithmetic operators (`+` , `-` , `*` , `/` , `**`). These methods automatically propagate coherence and history. For example, `C = A * B` results in a new `CoherenceState` `C` where `C.value = A.value * B.value` and `C.log_nrci = A.log_nrci + B.log_nrci` .\n\n* **Key Methods**\n * `sqrt()` : Returns the square root as a new `CoherenceState` . Coherence is preserved by dividing the `log_nrci` by 2.\n * `sin()` , `cos()` , `tan()` : Trigonometric functions that return new `CoherenceState` objects. These operations are treated as coherence-preserving for simplicity, though advanced studies may refine this.\n\n**Key Functions within the**

Module\n\n* **integrate(func, a, b, steps=1000)** : Numerically integrates a function `func` from `a` to `b`. Both `a` and `b` must be `CoherenceState` objects. The function `func` must accept and return `CoherenceState` objects. The resulting integral is a `CoherenceState` whose coherence is the average of the coherences of the intermediate steps.\n\n* **root(func, initial_guess, tolerance=1e-10, max_iterations=100)** : Finds the root of a function `func` using Newton's method, adapted for `CoherenceState` objects.\n\n### `y_constants.py` and `system_constants.py` \n\nThese modules define the fundamental constants of the UBP universe. In UBP 3.5, all constants are themselves `CoherenceState` objects with perfect `nrci` of 1.0.\n\n**Example: Accessing a Coherence-Native Constant**\n `python\nfrom coherence_substrate import Y_CONSTANT\nfrom system_constants import UBPConstants\n\n# Y_CONSTANT is a CoherenceState with value 0.264... and nrci=1.0\nprint(f"Y Constant: {Y_CONSTANT}")\n\n# O_OBSERVER is a CoherenceState with value 3.778... and nrci=1.0\nO_OBSERVER = UBPConstants.O_OBSERVER\nprint(f"Observer Cost: {O_OBSERVER}")\n` \n\n### `geometric_error_correction.py` (Unified Error Correction)\n\nThis module replaces the multiple, complex error correction modules of UBP 3.4. It provides a single, unified interface for maintaining and restoring coherence.\n\n* **restore_coherence(state: CoherenceState, reference: CoherenceState)** : The primary function for self-healing. It takes a degraded `state` and a `reference` state (typically a universal constant like `Y_CONSTANT`) and uses their geometric relationship to calculate a restored state with higher coherence. It returns a tuple containing the `(restored_state, correction_details)`.\n\n### `advanced_modules/field_dynamics.py` (Emergent Complexity)\n\nThis module is the successor to `CARFE` and demonstrates the power of the UBP 3.5 paradigm. It models complex physical field dynamics as an emergent property of the coherence substrate.\n\n* **recursive_evolution(initial_field: list, steps: int)** : Takes an `initial_field` (a list of `CoherenceState` objects) and simulates its evolution over a number of `steps`. Each step involves geometric transformations and coherence restoration, modeling the self-organizing behavior of a physical field.\n\n* **zitterbewegung(state: CoherenceState, frequency: float, cycles: int)** : Models the phenomenon of Zitterbewegung (rapid trembling motion) by applying high-frequency sinusoidal transformations to a `CoherenceState`, demonstrating how such complex dynamics can be handled natively.\n

---

7. Realm Operations in a Coherence-Native World {#realms}

The nine physical realm modules remain a core part of the UBP framework, providing the bridge between the abstract computational principles and concrete physical phenomena. In UBP 3.5, all realm modules have been refactored to operate natively with `CoherenceState` objects, offering deeper insights into the quality and integrity of physical calculations.

When you perform a calculation in a realm, the inputs (like frequency or coherence targets) are converted into `CoherenceState` objects, and the entire calculation is performed within the coherence substrate. The final result is also a `CoherenceState`, allowing you to inspect not just the final energy value but also the coherence of the calculation itself.

Example: Quantum Realm SOC Calculation

Let's compare a quantum energy calculation in UBP 3.4 and UBP 3.5 to highlight the change.

UBP 3.4 (Legacy):

Python

```python
# UBP 3.4 - Operates on floats
from quantum_realm import QuantumRealm, QuantumState

realm = QuantumRealm()
state = QuantumState(amplitude=1.0, phase=0.0, coherence=0.999997)

# Result is a dictionary of floats
result = realm.calculate_quantum_energy_soc(quantum_state=state,
frequency=2.466e15)
energy = result['energy_cu']
print(f"Energy: {energy:.6e} CU")
```

UBP 3.5 (Coherence-Native):

Python

```python
# UBP 3.5 - Operates on CoherenceState objects
from coherence_substrate import CoherenceState
from quantum_realm import QuantumRealm

realm = QuantumRealm()

# Inputs are CoherenceState objects
frequency = CoherenceState(2.466e15, nrci=1.0, history="Lyman-alpha freq")
initial_coherence = CoherenceState(0.999997, nrci=1.0, history="Target
Coherence")

# Result is a CoherenceState object
energy_state = realm.calculate_quantum_energy_soc(frequency,
initial_coherence)

print(f"Energy State: {energy_state}")
```

```
print(f"Final Energy Value: {energy_state.value:.6e} CU")
print(f"Coherence of Calculation: {energy_state.nrci:.6f}")
```

This demonstrates the richer, more informative output of the UBP 3.5 system. You get not only the answer but also a direct measure of its computational quality.

The Nine Realms of UBP 3.5

All nine realms are fully implemented and validated in UBP 3.5. They all follow the same coherence-native principles.

1. **quantum_realm.py** : Models quantum tunneling and state energy.
2. **atomic_realm.py** : Models spectroscopy and molecular vibrations.
3. **electromagnetic_realm.py** : Models antenna resonance and field energy.
4. **optical_realm.py** : Models visible spectrum phenomena and laser coherence.
5. **nuclear_realm.py** : Models binding energy and lattice dynamics.
6. **gravitational_realm.py** : Models gravitational waves and orbital resonance energy.
7. **biological_realm.py** : Models neural oscillations and DNA breathing modes.
8. **plasma_realm.py** : Models fusion reactions and stellar corona dynamics.
9. **cosmological_realm.py** : Models CMB fluctuations and Hubble expansion.

---

8. Advanced Features: Emergent Dynamics {#advanced}

The true power of the UBP 3.5 paradigm is revealed in its ability to model complex physical phenomena not as special cases with dedicated code, but as emergent properties of the underlying coherence substrate. The `advanced_modules/field_dynamics.py` module is the primary showcase for this capability.

Field Dynamics: The Successor to CARFE

In UBP 3.4, modeling the recursive evolution of a field required the `CARFE` module—a complex, 800+ line file with its own p-adic calculators and geometric engines. In UBP 3.5, this functionality is achieved with far greater elegance and power in the `field_dynamics.py` module.

This module demonstrates that phenomena like **recursive field evolution** and **Zitterbewegung** (trembling motion) are not special physical laws to be hard-coded, but are the natural consequence of applying fundamental geometric transformations (`*Y_CONSTANT`) and coherence restoration (`restore_coherence`) to a field of `CoherenceState` objects over time.

Recursive Field Evolution

The `recursive_evolution` function simulates how a field of values evolves under the influence of the UBP's geometric constraints. It is a loop that iteratively applies two simple rules:

1. **Transformation**: The field is transformed by a geometric constant (e.g., multiplied by `Y_CONSTANT`). This represents the natural evolution of the system.
2. **Coherence Restoration**: The coherence of the transformed field is then restored using the `restore_coherence` function. This represents the system's inherent tendency to return to a state of high coherence.

Python

```python
from coherence_substrate import CoherenceState, Y_CONSTANT
from advanced_modules.field_dynamics import recursive_evolution

# 1. Create an initial field of CoherenceState objects
initial_field = [CoherenceState(i, nrci=0.95) for i in range(10)]
print(f"Initial Field (first element): {initial_field[0]}")

# 2. Evolve the field for 5 steps
evolved_field = recursive_evolution(initial_field, steps=5)

# 3. Observe the result
print(f"Evolved Field (first element): {evolved_field[0]}")
```

The output shows how the value and coherence of the field elements change over time, purely as a result of the interplay between geometric transformation and coherence restoration. This simple loop produces complex, life-like behavior that accurately models physical fields.

Zitterbewegung: Intrinsic Trembling Motion

The `zitterbewegung` function models the rapid, trembling motion intrinsic to fundamental particles. It does so by applying a high-frequency sinusoidal transformation to a `CoherenceState`. This demonstrates how the coherence substrate can natively handle complex, time-dependent dynamics without needing a separate physics engine. The oscillation is not a simulation; it is a direct manipulation of the `CoherenceState`'s value in a way that preserves its computational history and coherence.

---

9. Migration Guide: From UBP 3.4 to 3.5 {#migration}

Migrating from UBP 3.4 to 3.5 requires more than just changing import paths; it requires a shift in thinking. Because UBP 3.5 is a complete paradigm shift, scripts written for 3.4 will not work without modification. This guide provides a clear, step-by-step process for updating your code.

The Core Principle of Migration: Embrace the `CoherenceState`

The fundamental task of migration is to stop using standard floats and start using `CoherenceState` objects for all UBP-related calculations. Every value that represents a physical quantity or a UBP parameter should be wrapped in a `CoherenceState`.

Step 1: Update Your Imports

First, update your import statements to point to the new UBP 3.5 modules. Many old modules have been consolidated, so you will need to find the new location for the functionality you need.

UBP 3.4 Module	UBP 3.5 Equivalent	Notes
`y_constants`	`coherence_substrate`	Core constants like `Y_CONSTANT` are now in the substrate.
`glr_base`, `level_7_global_golay`	`geometric_error_correction`	All error correction is now unified.
`carfe`	`advanced_modules/field_dynamics`	`CARFE` is superseded by the new emergent dynamics module.

Step 2: Convert Inputs to `CoherenceState` Objects

Anywhere you pass a numerical value into a UBP function, you must first convert it into a `CoherenceState`. This gives the value the computational properties required to interact with the UBP 3.5 system.

Before (UBP 3.4):

Python

```python
from quantum_realm import QuantumRealm

realm = QuantumRealm()
```

```
# Input is a raw float
result = realm.calculate_electromagnetic_energy(frequency_hz=5.45e14)
```

After (UBP 3.5):

Python

```
from coherence_substrate import CoherenceState
from quantum_realm import QuantumRealm

realm = QuantumRealm()
# Input is a CoherenceState
frequency = CoherenceState(5.45e14, nrci=1.0, history="Green Light Freq")
result_state = realm.calculate_electromagnetic_energy(frequency)
```

Step 3: Handle `CoherenceState` Outputs

UBP 3.5 functions do not return raw floats or dictionaries of floats. They return `CoherenceState` objects. You must update your code to handle these objects and access their attributes to get the information you need.

Before (UBP 3.4):

Python

```
# Result is a dictionary
energy_value = result["energy_cu"]
```

After (UBP 3.5):

Python

```
# Result is a CoherenceState object
energy_value = result_state.value
coherence_of_calculation = result_state.nrci

print(f"Energy: {energy_value}, Coherence: {coherence_of_calculation}")
```

Example: Migrating a Full Calculation

Let's walk through the migration of a complete example.

Original UBP 3.4 Code:

Python

```python
from gravitational_realm import GravitationalRealm

realm = GravitationalRealm()

# LIGO GW150914
result = realm.calculate_gravitational_energy(
    frequency_hz=250.0,
    target_nrci=0.999997
)

print(f"Energy: {result["energy_cu"]:.6e} CU")
```

Migrated UBP 3.5 Code:

Python

```python
from coherence_substrate import CoherenceState
from gravitational_realm import GravitationalRealm

realm = GravitationalRealm()

# 1. Convert inputs to CoherenceState objects
frequency = CoherenceState(250.0, nrci=1.0, history="LIGO Peak Freq")
target_nrci = CoherenceState(0.999997, nrci=1.0, history="Target NRCI")

# 2. Call the function with the new objects
energy_state = realm.calculate_gravitational_energy(frequency, target_nrci)

# 3. Access the attributes of the returned CoherenceState
print(f"Final Energy Value: {energy_state.value:.6e} CU")
print(f"Coherence of Calculation: {energy_state.nrci:.6f}")
print(f"Calculation History: {energy_state.history}")
```

By following these steps, you can systematically update your existing UBP 3.4 code to take full advantage of the power, simplicity, and transparency of the new coherence-native paradigm in UBP 3.5.

---

10. API Reference {#api}

This section provides a quick reference for the key classes and functions in UBP 3.5.

coherence_substrate.CoherenceState

- `CoherenceState(value, nrci, log_nrci, history)` : Constructor.

- `.value` : The float value.
- `.nrci` : The Non-Random Coherence Index.
- `.log_nrci` : The natural log of the NRCI.
- `.history` : String describing the object's origin.
- `.sqrt()` : Returns the square root as a new `CoherenceState` .

`geometric_error_correction`

- `restore_coherence(state, reference)` : Restores the coherence of a `state` using a `reference` `CoherenceState` . Returns `(restored_state, details)` .

`advanced_modules.field_dynamics`

- `recursive_evolution(initial_field, steps)` : Evolves a list of `CoherenceState` objects.
- `zitterbewegung(state, frequency, cycles)` : Applies high-frequency oscillations to a `CoherenceState` .

Realm Modules (e.g., `quantum_realm`)

- `calculate_..._energy_soc(frequency, target_nrci)` : All realm energy calculation functions now accept and return `CoherenceState` objects.

---

11. Appendices {#appendices}

Appendix A: The UBP 3.5 Module Consolidation

The following table details which UBP 3.4 modules were consolidated or superseded in the UBP 3.5 release.

UBP 3.4 Modules	Status in UBP 3.5	Justification
`glr_base`, `level_7_global_golay`, `metrics`	**Superseded** by `geometric_error_correction`	Algorithmic error correction is replaced by inherent geometric correction.
`carfe`	**Superseded** by `advanced_modules/field_dynamics`	Complex, hard-coded dynamics are now an emergent property of the substrate.
`p_adic_correction`	**Removed**	P-adic methods are a subset of the more general geometric correction.
`numpy`, `scipy`	**Removed**	All functionality is now implemented in pure Python within the coherence substrate.

Appendix B: The Philosophy of the Substrate

> "The substrate is the system."

This is the guiding principle of UBP 3.5. It means that the fundamental layer of the framework is not just a set of tools but is the active environment in which computation happens. The rules of the substrate define the physics of the system.

> "Computation is coherence."

This principle states that a calculation is not merely the manipulation of numbers but is a process that inherently involves the management of coherence. A result is not just a value; it is a state of information with a measurable quality. In UBP 3.5, every operation is a testament to this principle.

End of UBP 3.5 Instruction Manual

For the latest updates and to review the source code, visit the official repository: https://github.com/DigitalEuan/UBP_Repo/tree/main/ubp_3.5